RadarSim to Digital Twin

Upgrade Guide

3 August 2026

Contents

Against: RadarSim v2.4.0 (28,000 LOC, 217 tests passing)
Target: Kritzinger Level 2 — Digital Shadow
Date: 2026-08-03

Read this first: the precondition

A digital twin is a virtual counterpart of **one specific physical machine**, kept in step with it by an automated data connection. The definition is about the connection, not the fidelity.

This has a consequence that no amount of coding changes:

> **You cannot make RadarSim a digital twin without access to a real radar's data.**

So before writing any code, answer one question:

Which radar?

Your situation	What you can build
You have live access to a real radar	A real digital shadow — the full path in this guide
You can get *recorded* data from a real radar	Everything except the live link. Still genuinely valuable
Neither	**Twin-readiness**: the complete seam, exercised with replay. Hardware plugs in later

The third case is the realistic default for a solo open-source project, and this guide is written so that it is a first-class outcome rather than a consolation prize. Phases 1 and 2 are identical in all three cases. Only Phase 3 needs hardware.

Realistic ceiling

Level	Data flow	Reachable?
1 — Digital Model	None automated	**Where you are now**
2 — Digital Shadow	Automated physical → digital	**The target of this guide**
3 — Digital Twin	Automated bidirectional	Out of scope — requires closed-loop control of real hardware

Level 3 means the simulation automatically reconfigures the physical radar (PRF, waveform, beam schedule) with no human in the loop. That is a control-system and certification problem, not a software one. Do not aim for it.

The one idea that reframes the whole project

Today, **the simulation is the product**. The PPI picture is what you deliver.

As a shadow, **the difference is the product**. Nobody looks at the simulated picture. They look at:

> "The real radar is reporting 5.2 dB less SNR than physics predicts, and the gap has been widening for eleven days."

That sentence is the entire value proposition. Every task below exists to make it possible and trustworthy.

The hard part nobody mentions

A residual mixes two things together:

```
observed − predicted  =  (real asset degradation)  +  (your model's error)
```

If your model is wrong by 3 dB and the radar has degraded by 2 dB, the residual tells you nothing. **Separating those two terms is the actual engineering problem of a digital shadow.**

This is why determinism (§1.3) and calibration (§3.2) are not housekeeping — they are the load-bearing tasks. And it is also why your project is unusually well placed: `radar_equation.py` validated against Skolnik's worked example means the model-error term starts small. Most projects attempting this have the plumbing and an untrustworthy model. You have the opposite problem, which is the better one.

Phase 1 — Make it a headless service

Goal: everything RadarSim does works with no GUI, reproducibly, and fails loudly. **Hardware needed:** none. **This phase is mandatory in all three scenarios.**

A shadow runs unattended, 24/7, on a machine with no screen. Right now half your system cannot.

1.1 Move tracking into the engine — S

Problem. `track_manager.update()` is called in exactly one place in the entire codebase: `src/ui/thread_manager.py:159`. `SimulationEngine` constructs a `TrackManager` at `engine.py:278` and never runs it.

Consequences: `headless.py` and `batch_run.py` produce detections and **zero tracks**. And it is a layering inversion — domain logic living in the presentation layer.

Fix.

1. Add a tracking stage at the end of `SimulationEngine.step()`, after detection.
2. Have `step()` return tracks alongside `DetectionResult` objects.
3. Reduce `thread_manager.py` to a consumer — it displays tracks, it does not create them.

Do not skip this detail: the GUI currently feeds the tracker `target.position` — ground truth. Feed it `measured_range_m` / `measured_azimuth_rad` instead. Tracking ground truth makes the EKF pointless and makes every tracking-error metric identically zero. Since the residual *is* the product, this single line would silently invalidate the entire system.

Acceptance: `python headless.py --config scenarios/ku_band_tws.yaml` prints confirmed tracks. The gap analysis already verified the tracker converges to 3/3 when fed correctly, so this is wiring, not algorithm work.

1.2 Unify scenario loading — S

Problem. Two incompatible schemas. `headless.py` has an inline `load_config_from_yaml` reading a single `target.range_m`; `src/io/scenario_loader.py` reads a multi-target, position-based schema. Running `ku_band_tws.yaml` through `headless.py` ignores target positions and defaults to 50 km → zero detections.

Fix. Delete the inline loader. Point `headless.py` and `batch_run.py` at `ScenarioLoader`.

While you are there: `scenario_loader.py` is at **0% test coverage** despite being the entry path for every shipped scenario. Add tests for the schema it accepts, and for what it does with a malformed file. Untested ingest code becomes a liability the moment external data arrives.

1.3 Determinism and seeding — S/M

Problem. Detection draws use global `np.random` with no seed interface.

Why this is not "Low priority". Reconciliation compares a stochastic simulation against real measurements. Without reproducible runs you cannot distinguish a model error from a lucky draw, which means you cannot do the residual decomposition described above. For a shadow, determinism is load-bearing.

Fix.

1. Thread a `numpy.random.Generator` through `SimulationEngine` and every stochastic model (Swerling in `rcs.py`, clutter in `clutter.py`, detection draws in `engine.py`).
2. Accept a master seed in scenario YAML and on the CLI.
3. Add a regression test: same seed + same scenario → byte-identical detection stream.

Prefer `default_rng(seed)` over `np.random.seed()`. Global state and a long-running service are a bad combination.

1.4 Fail loudly — S

Problem. The `*_AVAILABLE` pattern wraps **first-party** modules in `try/except ImportError`. A typo in `clutter.py` sets `CLUTTER_AVAILABLE = False` and the simulation runs quietly without clutter.

Why this becomes dangerous as a shadow. Today it yields a slightly wrong picture. As a shadow: the model silently predicts without clutter, reports a large residual against the real radar, and an engineer concludes the *radar* is faulty. You would send someone up a mast to fix a bug in your import statements.

Fix.

1. Log at `WARNING` on every failed first-party import, with the exception text.
2. Add strict mode (`RADARSIM_STRICT=1` or a config flag) that raises instead.
3. Make strict mode the default for headless and service execution; keep permissive for the GUI.
4. Emit the active capability set at startup so it appears in logs.

Phase 1 exit criteria

- ☐ Headless run produces confirmed tracks, from measured detections
- ☐ One scenario schema across GUI, headless and batch
- ☐ Same seed → identical output, enforced by a test
- ☐ Failed first-party import raises in strict mode
- ☐ `scenario_loader.py` and `recorder.py` above 0% coverage

Phase 2 — Build the seam

Goal: RadarSim can consume detections it did not generate, and report how they differ from its own prediction.
Hardware needed: none. This entire phase is testable with replay.

This is where twin-readiness is actually created.

2.1 Define the detection contract — `S`

Before any code, define the wire format. This becomes an external interface, and external interfaces are expensive to change.

Minimum fields:

Field	Notes
`schema_version`	**Mandatory.** None of your current formats have one
`radar_id`	Which physical asset
`timestamp_utc`	Source-side, not receive-side
`timestamp_monotonic`	For ordering and latency measurement
`range_m`, `azimuth_rad`, `elevation_rad`	Measured, with uncertainty if available
`snr_db`	The primary residual channel
`amplitude`, `doppler_hz`	Optional
`quality` / `validity`	Source-reported confidence

Add `schema_version` to scenarios, HDF5 recordings and `.pkl` models at the same time. They are all unversioned today, and once a live feed exists, silent format drift becomes a production failure mode rather than a nuisance.

2.2 `MeasurementSource` abstraction — `M`

An abstract base with three implementations:

Implementation	Source	Purpose
`SimulatedSource`	Current engine physics	Wraps today's behaviour — no regression
`ReplaySource`	HDF5 recordings, CSV	**Build and test everything with this**
`LiveSource`	Network / hardware	Phase 3

The engine consumes detections from a source rather than always generating them. `SimulatedSource` must produce byte-identical results to v2.4.0 for the existing 217 tests — that is your safety net for the refactor.

`ReplaySource` is the important one. It lets you build and validate the entire ingest, timing and reconciliation stack with zero hardware. When a real radar becomes available, you swap one class.

2.3 Time base — `M`

The gap register lists this as missing but no roadmap step addresses it. Residuals between a live feed and a sim on an internal clock are meaningless.

Decide and implement:

- **Free-run** — sim advances on its own clock (today's behaviour, keep for GUI)
- **Locked** — sim advances to match source timestamps (required for shadow mode)

Then handle the unglamorous cases, because real feeds do all of them:

- Out-of-order arrivals → bounded reorder buffer
- Dropouts → explicit gap markers, never silent interpolation
- Clock skew between source and host → measure and report it, don't correct silently

- Latency → track it; it belongs in the health output

2.4 Reconciliation — `M/L`

The product. For each observed detection, pair it against the model's prediction for the same target-geometry and emit:

- `Δ SNR (dB)` — the primary health channel
- `Δ range`, `Δ azimuth` — geometry and calibration errors
- `Δ Pd` over a window — detection-performance drift
- Track-level: position RMSE, track continuity, false-track rate

Report rolling statistics, not per-frame values. A single-frame residual is dominated by Swerling fluctuation and tells you nothing. A 24-hour rolling mean with a confidence interval tells you the transmitter is dying.

Output:

1. A residual stream (append-only, HDF5 or Parquet)
2. A health summary (current deltas, trend, alert state)
3. Threshold alerts

Phase 2 exit criteria

- ☐ Versioned detection schema, documented
- ☐ `MeasurementSource` with all three implementations
- ☐ All 217 tests pass unchanged via `SimulatedSource`
- ☐ Replay of a recording produces a residual stream
- ☐ Injected fault (e.g. −4 dB on all detections) is detected and reported

At this point you can honestly write "digital-twin-ready architecture" in the README.

Phase 3 — Connect a real radar

Hardware required. Only reachable if you have an asset and its data.

3.1 Choose the interface — `S/M`

Driven entirely by the radar, not by you. Likely candidates: ASTERIX (Cat 48 for target reports, Cat 34 for service messages) for air-surveillance sets; a vendor TCP/UDP protocol; or a CSV/log tail for a research set. Implement `LiveSource` against whatever exists. Everything downstream is already built and tested from Phase 2.

3.2 Calibrate the model to *that* radar — `L`

This is the step that makes it a twin rather than a simulation with a data feed, and it is the one most likely to be underestimated.

Generic parameters must be replaced with measured properties of the specific unit:

- Measured antenna pattern, not `sinc` or approximate Taylor
- Actual system losses, not a nominal figure
- Measured receiver noise figure
- Actual transmit power at the flange
- Real PRF, dwell and scan schedule
- Site-specific terrain and clutter environment

Then fit residual model parameters against a period of known-good operation until the mean residual is near zero.

Note the interaction with the fidelity backlog: §2.2 of your uploaded review (no `G(az,el)`, circular symmetry assumed) is not a twin blocker in the abstract — but it *becomes* one here, because a real antenna has different azimuth and elevation beamwidths and you cannot calibrate to it with a symmetric pattern. **This is the point at which selected fidelity work becomes necessary, and it is driven by the specific radar rather than by a generic wish-list.**

3.3 Establish a healthy baseline — `M`

Run against the real radar for a sustained period of known-good operation. This produces the reference distribution of residuals. Without it, a residual is a number with no meaning.

3.4 Alerting — `S`

Alert on *deviation from baseline*, not on absolute values. Use trend and dwell time rather than instantaneous thresholds — you want "mean Δ SNR has exceeded 2σ for 6 hours", not "one frame was low".

Sequence and effort

#	Task	Phase	Effort	Hardware	Blocks
1	Tracking into `engine.step()`, measured input	1	S	No	Everything
2	Unify on `ScenarioLoader` + tests	1	S	No	Repeatable runs
3	Seeding and determinism	1	S/M	No	Meaningful residuals
4	Loud failures + strict mode	1	S	No	Trustworthy operation
5	Detection schema + versioning	2	S	No	The seam
6	`MeasurementSource` (sim/replay/live)	2	M	No	Ingest
7	Time base and sync	2	M	No	Comparison
8	Reconciliation and residuals	2	M/L	No	**The product**
9	`LiveSource` for a real radar	3	S/M	**Yes**	Shadow status
10	Calibrate to the specific unit	3	L	**Yes**	Twin-ness
11	Baseline and alerting	3	M	**Yes**	Operational use

Tasks 1–8 are roughly **one to three months part-time** for one developer. They require no hardware and leave the project genuinely twin-ready.

What not to do

Do not work the fidelity backlog first. The uploaded upgrade summary lists NLFM, OS-CFAR, MIMO, GPU, Keystone transform and IMM/JPDA. Every one is a fidelity improvement. None is a twin capability. You could implement all of them perfectly and have zero digital-shadow capability.

Also be aware that document reviewed 5 files out of ~60 and its "CRITICAL" items — radar equation, ITU-R P.676, Swerling, unified engine, pytest suite — all already exist in your code. Its genuine findings are narrower: NLFM and OS-CFAR declared but unimplemented, Taylor pattern is a sidelobe cap, no `G(az,el)`, `rf_spectrum.py` LPI is a toy, MIMO and IMM/JPDA absent.

Handle those as a **separate backlog**, and pull from it only when Phase 3 calibration demands it.

Do not claim "digital twin" in the README. Use "digital-twin-ready measurement pipeline" or "twin-capable architecture". Radar engineers will respect the precision and distrust the overclaim. The project's strongest credibility asset is physics validated against Skolnik to a tight tolerance — do not spend it on a marketing word.

Do not add GPU acceleration before the seam. It optimises a path you are about to restructure.

Honesty checklist for the README

Fix these regardless of the twin work, because they are correctness issues in what the project *claims*:

- `WaveformType.NLFM` — enum value, no implementation
- `CFARType.OS` — enum value, no implementation
- `taylor_pattern()` — a sinc with a sidelobe cap; docstring admits "approximate", the README does not
- `src/tracking/monopulse.py` — orphaned, imported nowhere, yet advertised as "sub-beamwidth accuracy"
- `src/signal/rf_spectrum.py`, `src/components/antenna.py` — orphaned
- `src/advanced/simulation_engine.py` + `webgl_renderer.py` — ~450 statements of dead code, a second `SimulationEngine` in the tree
- `pyproject.toml` — declares PyQt6 (24 files import PySide6), lists unused `pygame`, console script points at a non-existent `main.py`
- `CITATION.cff` says version 1.0.0; project is 2.4.0

Where this leaves you

Physics: ready. This is the hard part and it is done. **System:** not ready. Phases 1 and 2 fix it, no hardware needed. **Hardware:** absent. Not a coding problem.

The honest one-line status, now and after each phase:

Stage	Accurate description
Today	Physics-based radar simulator (Digital Model)
After Phase 1	Headless, reproducible radar simulation service
After Phase 2	Digital-twin-ready simulation with replay-validated ingest
After Phase 3	Digital shadow of [*named radar*]

Aim for row 3. It is fully within reach on your own, it is the honest maximum without hardware, and it is the state that makes row 4 a swap of one class rather than a rewrite.